

API de Mesa de Ayuda (Helpdesk) con SLA - Spring Boot + JWT

1. Contexto

Una empresa necesita un sistema sencillo para gestionar **tickets de soporte técnico**. Los usuarios reportan incidencias y el equipo de soporte las atiende. Cada ticket tiene una **prioridad** que define un **SLA** (tiempo máximo de respuesta comprometido). El sistema debe controlar quién puede hacer qué según su rol. Usted desarrollará la **API REST** que soporta este sistema. No se requiere frontend.

2. Objetivos de aprendizaje

Al finalizar el taller, usted estará en capacidad de:

- Construir una API REST con Spring Boot y Spring Data JPA.
 - Implementar autenticación con **JWT** (registro y login).
 - Implementar una estrategia de **refresh token** para renovar el acceso sin volver a autenticarse.
 - Proteger rutas mediante **autenticación y autorización por rol** (RBAC).
 - Diferenciar rutas públicas de rutas protegidas.
 - Aplicar reglas de negocio (cálculo de SLA) del lado del servidor.
-

3. Stack tecnológico

Componente	Tecnología
Lenguaje	Java 17+
Framework	Spring Boot 3.x
Seguridad	Spring Security + JWT

Componente	Tecnología
Persistencia	Spring Data JPA
Base de datos	H2 (en memoria), SQLite, MySQL o PostgreSQL
Pruebas	Postman, Thunder Client, Insomnia, Bruno, etc

Dependencias sugeridas: `Spring Web`, `Spring Security`, `Spring Data JPA`, `H2 Database` (o `Driver correspondiente`), `Lombok`, `Validation`, `jjwt`.

4. Modelo de datos

Entidad `Usuario`

Campo	Tipo	Notas
id	Long	PK, autogenerado
nombre	String	Obligatorio
email	String	Único, obligatorio
password	String	Almacenado cifrado (BCrypt)
rol	Enum	<code>USUARIO</code> , <code>SOPORTE</code> , <code>ADMIN</code>

Entidad `Ticket`

Campo	Tipo	Notas
id	Long	PK, autogenerado
titulo	String	Obligatorio
descripcion	String	Obligatorio
prioridad	Enum	<code>BAJA</code> , <code>MEDIA</code> , <code>ALTA</code>
estado	Enum	<code>ABIERTO</code> , <code>EN_PROCESO</code> , <code>RESUELTO</code> (por defecto <code>ABIERTO</code>)
creadoEn	DateTime	Fecha de creación (automática)
slaVenceEn	DateTime	Calculado según prioridad (ver regla)
creadoPor	Usuario	Relación con el usuario que lo creó

5. Regla de negocio: cálculo del SLA

Al **crear** un ticket, el servidor debe calcular automáticamente `slaVenceEn` sumando a la fecha de creación las siguientes horas según la prioridad:

Prioridad	SLA (horas)
ALTA	4 horas
MEDIA	24 horas
BAJA	72 horas

El cliente **nunca** envía `slaVenceEn` ni `estado`; ambos los define el servidor.

Adicionalmente, exponga un campo o endpoint que indique si un ticket está **vencido** (la fecha actual superó `slaVenceEn` y el estado no es `RESUELTO`).

6. Estrategia de autenticación: Access Token + Refresh Token

El sistema usa **dos tokens** con propósitos distintos:

Token	Vida útil sugerida	Uso
Access token (JWT)	Corta (ej. 15 min)	Se envía en cada petición para acceder a rutas protegidas.
Refresh token	Larga (ej. 7 días)	Sirve únicamente para solicitar un nuevo access token cuando el anterior expira.

6.1. Flujo esperado

1. El usuario hace **login**, el servidor devuelve `accessToken` + `refreshToken`.
2. El cliente usa el `accessToken` en el header `Authorization` hasta que expira.
3. Cuando el `accessToken` expira (401), el cliente llama a `/api/auth/refresh` enviando el `refreshToken`.
4. Si el `refreshToken` es válido y no está revocado, el servidor emite un **nuevo** `accessToken` (y opcionalmente un nuevo `refreshToken`, ver *rotación* abajo).

5. En el **logout**, el `refreshToken` se **revoca** para que no pueda reutilizarse.

6.2. ¿Dónde persistir el refresh token?

Usted decide la estrategia. Se sugieren dos enfoques, del más simple al más robusto:

Opción A - Persistido en base de datos (recomendado para este taller)

Cree una entidad `RefreshToken`:

Campo	Tipo	Notas
id	Long	PK
token	String	El valor del refresh token (o su hash). Único
usuario	Usuario	A quién pertenece
expiraEn	DateTime	Fecha de expiración
revocado	Boolean	<code>true</code> si ya no es válido (logout o rotación)

Ventajas: permite **revocar** tokens (logout real), invalidar sesiones y llevar control. Es la opción más didáctica porque obliga a razonar sobre el ciclo de vida del token.

Opción B - Refresh token también como JWT (stateless)

El refresh token es otro JWT firmado, con mayor expiración y un `claim` que lo distingue del access token. No se guarda en DB.

Ventajas: no requiere tabla ni consultas. Desventaja: **no se puede revocar** fácilmente (el logout sería solo del lado del cliente). Si eligen esta opción, expliquen en el README esa limitación.

Requisito mínimo: sea cual sea la opción, el **logout debe invalidar** el refresh token. Con la Opción A esto es directo (marcar `revocado = true`); con la Opción B deben proponer un mecanismo (ej. una *denylist* en memoria/DB).

7. Endpoints requeridos

7.1. Rutas públicas (sin autenticación)

Método	Ruta	Descripción
POST	/api/auth/registro	Registra un nuevo usuario con rol USUARIO
POST	/api/auth/login	Autentica y devuelve accessToken + refreshToken
POST	/api/auth/refresh	Recibe un refreshToken válido y devuelve un nuevo accessToken
GET	/api/ping	Verifica que la API está viva (responde pong)

7.2. Rutas protegidas por autenticación (cualquier usuario logueado)

Método	Ruta	Descripción
POST	/api/auth/logout	Revoca el refreshToken del usuario autenticado
POST	/api/tickets	Crea un ticket (el creador es el usuario autenticado)
GET	/api/tickets/mios	Lista los tickets creados por el usuario autenticado
GET	/api/tickets/{id}	Consulta un ticket (solo si es el dueño o tiene rol SOPORTE/ADMIN)

7.3. Rutas protegidas por rol

Método	Ruta	Rol requerido	Descripción
GET	/api/tickets	SOPORTE , ADMIN	Lista todos los tickets
PATCH	/api/tickets/{id}/estado	SOPORTE , ADMIN	Cambia el estado de un ticket
GET	/api/tickets/vencidos	SOPORTE , ADMIN	Lista los tickets que superaron su SLA
POST	/api/admin/soporte	ADMIN	Asciende a un usuario existente al rol SOPORTE

8. Requisitos de seguridad

1. Las contraseñas deben almacenarse cifradas con **BCrypt**. Nunca en texto plano.
2. El login exitoso devuelve un **access token (JWT)** que incluye el email y el rol del usuario, más un **refresh token**.
3. Toda ruta protegida exige el header `Authorization: Bearer <accessToken>`.
4. Si el token falta o es inválido, responder **401 Unauthorized**.
5. Si el usuario está autenticado pero no tiene el rol necesario, responder **403 Forbidden**.
6. Un usuario con rol `USUARIO` **no** puede ver tickets de otros usuarios.
7. Un `refreshToken` expirado, revocado o inexistente en `/api/auth/refresh` debe responder **401 Unauthorized**.
8. El `refreshToken` **no** debe servir para acceder a rutas protegidas; solo para renovar el access token.

9. Validaciones mínimas

- `email` debe tener formato válido y ser único.
- `password` con longitud mínima de 6 caracteres.
- `titulo` y `descripcion` no pueden estar vacíos.
- `prioridad` y `estado` solo aceptan los valores del enum; en caso contrario responder **400 Bad Request**.

10. Códigos de respuesta esperados

Situación	Código
Creación exitosa	201 Created
Consulta exitosa	200 OK
Datos inválidos	400 Bad Request
Sin token / token inválido / refresh inválido	401 Unauthorized
Rol insuficiente	403 Forbidden

Situación	Código
Recurso no encontrado	404 Not Found
Email ya registrado	409 Conflict

11. Entregables

1. Repositorio Git con el código fuente y un `README.md` con instrucciones de ejecución.
2. En el `README.md`, indique **qué estrategia de persistencia de refresh token** eligió (Opción A o B) y por qué.
3. Colección de **Postman o cliente rest utilizado o Swagger** que pruebe todos los endpoints.
4. Evidencia, video demostrando:
 - Registro y login con obtención del `accessToken` y `refreshToken`.
 - Renovación del access token vía `/api/auth/refresh`.
 - Logout que revoca el refresh token (un `/refresh` posterior falla con 401).
 - Acceso denegado (401) a una ruta protegida sin token.
 - Acceso denegado (403) a una ruta de rol con un usuario `USUARIO`.
 - Creación de un ticket con SLA calculado correctamente.
 - Listado de tickets vencidos por un usuario `SOPORTE`.

12. Rúbrica de evaluación

Criterio	Peso
API funcional y modelo de datos correcto	15%
Autenticación con JWT (registro/login)	15%
Estrategia de refresh token (emisión, renovación y revocación)	20%
Protección de rutas por autenticación	10%
Autorización por rol (RBAC)	20%

Criterio	Peso
Cálculo correcto del SLA y tickets vencidos	10%
Validaciones, códigos HTTP y buenas prácticas	10%

13. Reto opcional (bono)

- Agregar paginación al listado general de tickets.
- Registrar un historial de cambios de estado por ticket (admin, o soporte y usuarios relacionados al ticket).
- Endpoint de estadísticas: cantidad de tickets por estado y % de cumplimiento de SLA (solo admin).